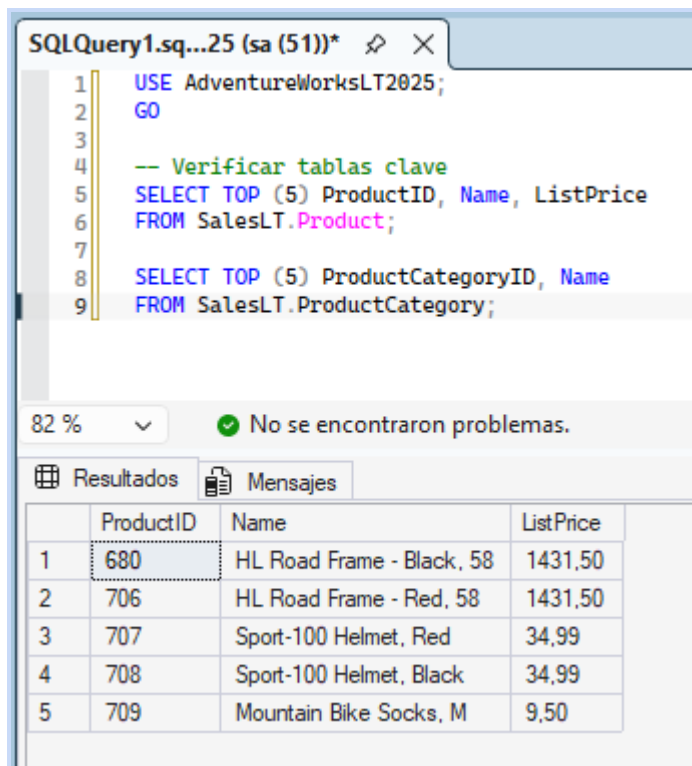


Laboratorio 03 - Write advanced T-SQL queries

- Connect to AdventureWorksLT

- Verificamos la conexión a la base de datos **AdventureWorksLT2025** y comprobamos el acceso a las tablas principales de productos y categorías.



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a T-SQL query in the 'SQLQuery1.sql...25 (sa (51))*' window. The query is as follows:

```
1 USE AdventureWorksLT2025;
2 GO
3
4 -- Verificar tablas clave
5 SELECT TOP (5) ProductID, Name, ListPrice
6 FROM SalesLT.Product;
7
8 SELECT TOP (5) ProductCategoryID, Name
9 FROM SalesLT.ProductCategory;
```

Below the query editor, the status bar indicates '82 %' zoom and 'No se encontraron problemas.' (No problems found).

The bottom pane shows the 'Resultados' (Results) tab, displaying a table with 5 rows and 4 columns: ProductID, Name, ListPrice, and an additional column (likely ProductCategoryID). The data is as follows:

	ProductID	Name	ListPrice	
1	680	HL Road Frame - Black, 58	1431,50	
2	706	HL Road Frame - Red, 58	1431,50	
3	707	Sport-100 Helmet, Red	34,99	
4	708	Sport-100 Helmet, Black	34,99	
5	709	Mountain Bike Socks, M	9,50	

- **Build JSON output from product data**

- Convertimos filas de productos en objetos JSON con **FOR JSON PATH** y creamos estructuras anidadas de categorías usando **JSON_OBJECT**.

1. **Crear un objeto JSON simple por cada producto (FOR JSON PATH).**

```
SQLQuery2.sql...25 (sa (64))* connect-adven...025 (sa
1 SELECT
2     ProductID,
3     Name,
4     Color,
5     ListPrice
6 FROM SalesLT.Product
7 WHERE Color IS NOT NULL
8 ORDER BY ListPrice DESC
9 FOR JSON PATH;
```

132 % No se encontraron problemas.

Resultados Mensajes

	JSON_F52E2B61-18A1-11d1-B105-00805F49916B
1	[{"ProductID":749,"Name":"Road-150 Red, 62","Col...

2. **Crear objetos JSON anidados con la categoría del producto (JSON_OBJECT).**

```
SQLQuery3.sql...25 (sa (53))* create-JSON.s...2025 (sa (64)) connect-adven...025 (sa (51))
1 SELECT
2     p.ProductID,
3     p.Name AS ProductName,
4     p.ListPrice,
5     JSON_OBJECT(
6         'CategoryID' : pc.ProductCategoryID,|
7         'CategoryName' : pc.Name
8     ) AS Category
9 FROM SalesLT.Product AS p
10 INNER JOIN SalesLT.ProductCategory AS pc
11     ON p.ProductCategoryID = pc.ProductCategoryID
12 ORDER BY p.ListPrice DESC
13 FOR JSON PATH;
```

132 % No se encontraron problemas. Línea: 6, C

Resultados Mensajes

	JSON_F52E2B61-18A1-11d1-B105-00805F49916B
1	[{"ProductID":749,"ProductName":"Road-150 Red, 6...

- Combine JSON with a CTE and window function

- Clasificamos los productos más caros por categoría mediante una CTE con **ROW_NUMBER()** y exportamos el resultado envuelto en un objeto raíz **TopProducts**.

1. Crear la consulta con CTE y ROW_NUMBER().

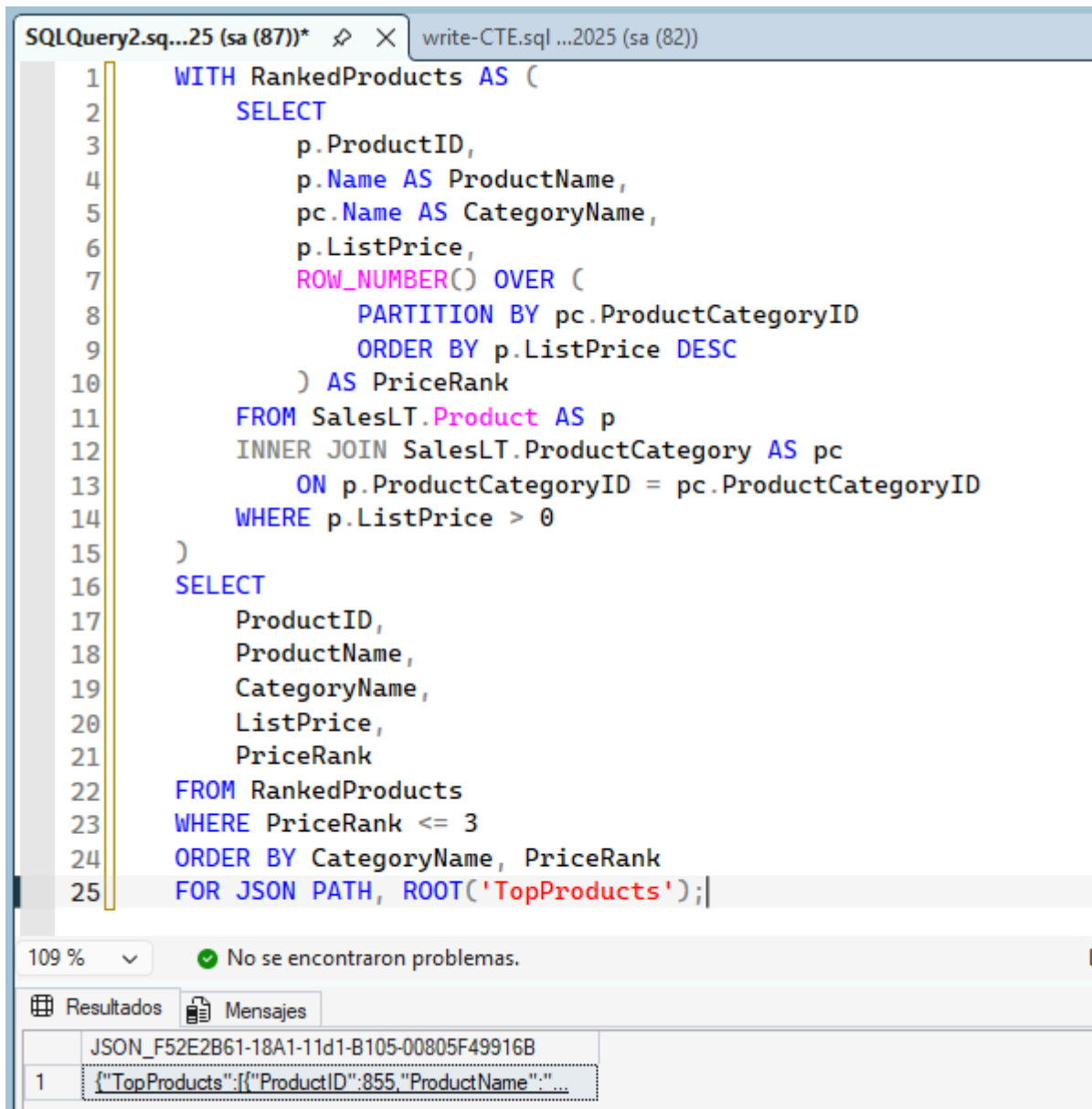
The screenshot shows a SQL query in the 'SQLQuery1.sql' window. The query uses a Common Table Expression (CTE) named 'RankedProducts' to rank products by price within each category. The final query selects the top 3 products from each category based on their price rank.

```
1  USE AdventureWorksLT2025;
2  GO
3
4  WITH RankedProducts AS (
5      SELECT
6          p.ProductID,
7          p.Name AS ProductName,
8          pc.Name AS CategoryName,
9          p.ListPrice,
10         ROW_NUMBER() OVER (
11             PARTITION BY pc.ProductCategoryID
12             ORDER BY p.ListPrice DESC
13         ) AS PriceRank
14     FROM SalesLT.Product AS p
15     INNER JOIN SalesLT.ProductCategory AS pc
16         ON p.ProductCategoryID = pc.ProductCategoryID
17     WHERE p.ListPrice > 0
18 )
19 SELECT
20     ProductID,
21     ProductName,
22     CategoryName,
23     ListPrice,
24     PriceRank
25 FROM RankedProducts
26 WHERE PriceRank <= 3
27 ORDER BY CategoryName, PriceRank;
28 GO
```

Below the query editor, the 'Resultados' (Results) tab is active, displaying the query output as a table. The status bar indicates 'No se encontraron problemas.' (No problems found).

	ProductID	ProductName	CategoryName	ListPrice	PriceRank
1	855	Men's Bib-Shorts, S	Bib-Shorts	89,99	1
2	856	Men's Bib-Shorts, M	Bib-Shorts	89,99	2
3	857	Men's Bib-Shorts, L	Bib-Shorts	89,99	3
4	876	Hitch Rack - 4-Bike	Bike Racks	120,00	1
5	879	All-Purpose Bike Stand	Bike Stands	159,00	1
6	871	Mountain Bottle Cage	Bottles and Cages	9,99	1
7	872	Road Bottle Cage	Bottles and Cages	8,99	2
8	870	Water Bottle - 30 oz.	Bottles and Cages	4,99	3
9	996	HL Bottom Bracket	Bottom Brackets	121,49	1
10	995	ML Bottom Bracket	Bottom Brackets	101,24	2
11	994	LL Bottom Bracket	Bottom Brackets	53,99	3

2. Formatear el resultado como un JSON con elemento raíz (ROOT).



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a T-SQL query in the 'SQLQuery2.sql' file. The query uses a Common Table Expression (CTE) named 'RankedProducts' to rank products by list price within each product category. The final output is formatted as JSON using the 'FOR JSON PATH, ROOT('TopProducts');' clause.

```
1 WITH RankedProducts AS (  
2     SELECT  
3         p.ProductID,  
4         p.Name AS ProductName,  
5         pc.Name AS CategoryName,  
6         p.ListPrice,  
7         ROW_NUMBER() OVER (  
8             PARTITION BY pc.ProductCategoryID  
9             ORDER BY p.ListPrice DESC  
10        ) AS PriceRank  
11     FROM SalesLT.Product AS p  
12     INNER JOIN SalesLT.ProductCategory AS pc  
13         ON p.ProductCategoryID = pc.ProductCategoryID  
14     WHERE p.ListPrice > 0  
15 )  
16 SELECT  
17     ProductID,  
18     ProductName,  
19     CategoryName,  
20     ListPrice,  
21     PriceRank  
22 FROM RankedProducts  
23 WHERE PriceRank <= 3  
24 ORDER BY CategoryName, PriceRank  
25 FOR JSON PATH, ROOT('TopProducts');
```

The bottom pane shows the execution results. The status bar indicates 'No se encontraron problemas.' (No problems were found). The 'Resultados' (Results) tab is active, displaying a single row of JSON data:

	JSON_F52E2B61-18A1-11d1-B105-00805F49916B
1	{ "TopProducts": [{ "ProductID": 855, "ProductName": "...

- **Parse JSON data with OPENJSON**

- Leímos una cadena JSON con **OPENJSON** para transformarla en una tabla relacional y la combinamos con la tabla **Product** para calcular la diferencia de precios.

1. **Convertir una cadena JSON en filas.**

The screenshot shows a SQL query in SQL Server Enterprise Manager. The query uses the **OPENJSON** function to parse a JSON string into a table. The JSON string contains three objects, each representing a product update with a **ProductID** and a **NewPrice**. The query declares a variable **@ProductUpdates** containing the JSON string, then selects the **ProductID** and **NewPrice** columns from the parsed JSON. The results are displayed in a table with three rows.

```
SQLQuery3.sql...25 (sa (89))*  json-path.sql ...2025 (sa (87)) write-  
1 DECLARE @ProductUpdates NVARCHAR(MAX) = N'  
2     {"ProductID": 680, "NewPrice": 1250.00},  
3     {"ProductID": 706, "NewPrice": 1450.00},  
4     {"ProductID": 707, "NewPrice": 38.99}  
5 ]';  
6  
7 SELECT  
8     ProductID,  
9     NewPrice  
10 FROM OPENJSON(@ProductUpdates)  
11 WITH (  
12     ProductID INT '$.ProductID',  
13     NewPrice DECIMAL(10,2) '$.NewPrice'  
14 );
```

109 % No se encontraron problemas.

Resultados Mensajes

	ProductID	NewPrice
1	680	1250.00
2	706	1450.00
3	707	38.99

2. Combinar el JSON parseado con la tabla de productos (JOIN).

SQLQuery4.sql...25 (sa (79))* json-array-ro...2025 (sa (89)) json-path.sql ...2025

```
4      {"ProductID": 707, "NewPrice": 38.99}
5    ]';
6
7    SELECT
8      p.ProductID,
9      p.Name,
10     p.ListPrice AS CurrentPrice,
11     updates.NewPrice,
12     updates.NewPrice - p.ListPrice AS PriceDifference
13 FROM SalesLT.Product AS p
14 INNER JOIN OPENJSON(@ProductUpdates)
15 WITH (
16     ProductID INT '$.ProductID',
17     NewPrice DECIMAL(10,2) '$.NewPrice'
18 ) AS updates
19 ON p.ProductID = updates.ProductID;
```

109 % No se encontraron problemas.

Resultados Mensajes

	ProductID	Name	CurrentPrice	NewPrice	PriceDifference
1	680	HL Road Frame - Black, 58	1431,50	1250.00	-181.5000
2	706	HL Road Frame - Red, 58	1431,50	1450.00	18.5000
3	707	Sport-100 Helmet, Red	34,99	38.99	4.0000